

Reviewer Pack — Forman-Ricci Regularity Bound Certifies Correct F^*_v Minterm...

Entry e90d653a85bf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:51:03 UTC

Forman-Ricci Regularity Bound Certifies Correct F^*_v Minterms

Entry ID: e90d653a85bf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:51:03 UTC

1. Conjecture

Field A (mathematical branch): Forman-Ricci curvature **Field B** (complexity object): Monotone Boolean functions

Statement:

Let $v \in \{4, 10, 16, 20\}$ and $k = \lfloor \log_2 v \rfloor$ ($k=2$ for $v=4$, $k=4$ for $v=10,16$, $k=5$ for $v=20$). Let F^*_v denote the k -clique minterm DNF of K_v whose minterms are the $C(v,k)$ edge-sets of k -vertex cliques (one minterm per k -subset of $[v]$). Let $G(F^*_v)$ be the simple unweighted graph whose nodes are the minterms of F^*_v and whose edges connect pairs of minterms that share exactly $k-1$ vertices. Conjecture: for every such v , (i) the minterm count satisfies $M(F^*_v) \geq C(v,k)$; and (ii) the average unweighted edge Forman-Ricci curvature $\bar{\kappa}(G(F_v)) := (1/|E(G)|) \cdot \sum_{\{u,w\} \in E(G)} (4 - \deg(u) - \deg(w))$ satisfies $|\bar{\kappa}(G(F_v)) - (4 - 2k(v-k))| \leq 1$. A single v for which either (i) fails or (ii) is violated refutes the conjecture.

Rationale (proposer's reasoning):

SC1 needs a single sharp combinatorial witness that distinguishes the correct $C(v,k)$ clique-minterm DNF from broken generators (e.g., star DNFs with only 2 terms or duplicated-vertex artifacts). When `generate_clique_dnf` is correct, $G(F^*_v)$ is exactly $k(v-k)$ -regular by Johnson-scheme combinatorics, making $\bar{\kappa}$ pinned at $4 - 2k(v-k)$; any incorrect generator destroys this regularity and the average curvature drifts outside the ± 1 envelope. The Forman-Ricci value thus acts as a one-number certificate that the F^*_v construction is being measured on the actual k -clique minterm lattice rather than a degenerate substitute.

Taxonomy category: `forman_ricci_dnf` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6b8fbb906e9138c3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each $v \in \{4, 10, 16, 20\}$ with $k = \lceil \log_2 v \rceil$, across 30 seeded random vertex relabelings, require every seed to satisfy $M(F^*_v) = C(v, k)$ AND $|\bar{k}(G) - (4 - 2k(v - k))| \leq 1$. Support iff all $4 \times 30 = 120$ (v, seed) trials pass both; a single failure falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman Ricci curvature Johnson graph - Forman curvature clique complex minterm Boolean - average Forman-Ricci curvature regular graph k-subset

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, permutations

def generate_clique_dnf(v):
    k = math.ceil(math.log2(v))
    cliques = list(combinations(range(v), k))
    minterms = [frozenset(clique) for clique in cliques]
    return minterms

def compute_degree(graph, node):
    return sum(1 for neighbor in graph[node] if neighbor != node)

def compute_forman_ricci_curvature(graph):
    n_edges = sum(len(neighbors) for neighbors in graph.values()) // 2
    curvature_sum = sum(4 - deg_u - deg_w for u in graph for w in graph[u] if u < w)
    return curvature_sum / (2 * n_edges)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for v in [4, 10, 16, 20]:
        k = math.ceil(math.log2(v))
        minterms = generate_clique_dnf(v)
```

```

# Relabel vertices randomly
relabeling = list(permutations(range(v)))
graph = {i: set() for i in range(len(minterms))}
for u, minterm in enumerate(minterms):
    for v in range(u + 1, len(minterms)):
        shared_edges = minterm & minterms[v]
        if len(shared_edges) == k - 1:
            graph[u].add(v)
            graph[v].add(u)

M_Fv = len(minterms)
predicted_bound = 4 - 2 * k * (v - k)
curvature = compute_forman_ricci_curvature(graph)

results.append({
    "metric_name": "M(F_v)",
    "metric_value": M_Fv,
    "instances_tested": len(minterms),
    "n_max": v,
    "conjecture_holds": M_Fv == C(v, k) and abs(curvature - predicted_bound) <= 1,
    "counterexample": ""
})

return {
    "seed": seed,
    "results": results
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    all_results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        all_results.extend(trial_result["results"])

    metric_values = [result["metric_value"] for result in all_results]
    conjecture_holds = all(result["conjecture_holds"] for result in all_results)

    if conjecture_holds:
        mean_metric = sum(metric_values) / len(metric_values)
        std_metric = (sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)) ** 0.5
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(all_results) if not result["conjecture_holds"])
        counterexample = all_results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_892b0af1.py", line 72, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_892b0af1.py", line 51, in run_trial
    curvature = compute_forman_ricci_curvature(graph)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_892b0af1.py", line 28, in compute_forman_ricci_curvature
    curvature_sum = sum(4 - deg_u - deg_w for u in graph for w in graph[u] if u < w)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_892b0af1.py", line 28, in <genexpr>
    curvature_sum = sum(4 - deg_u - deg_w for u in graph for w in graph[u] if u < w)
                   ^^^^^
```

NameError: name 'deg_u' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError (deg_u undefined) before producing any data, so neither the support nor falsification condition could be evaluated. | next: Fix the bug in compute_forman_ricci_curvature by computing deg_u = len(graph[u]) and deg_w = len(graph[w]) inside the comprehension, then rerun the 120 trials.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	128162
2	propose	claude_max	opus	0	0	85062
3	preregistration	claude_max	opus	0	0	5678
4	novelty	claude_max	opus	0	0	3370
5	novelty	claude_max	opus	0	0	7441
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20363
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15562
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15229
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9272
10	judge	claude_max	opus	0	0	3210

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 293349 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/e90d653a85bf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e90d653a85bf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e90d653a85bf.tar.gz` (if generated)